

FACULTAD DE INGENIERÍA

Taller de Tecnologías 2

Obligatorio

Turno N6A

Prof. Joaquín Bonifacino

Prof. Danilo Fraque

Autores:

Santiago Aguerre - 274715

Juan Croquis - 286049

Emiliano Reyes - 251525

Índice

Arquitectura del agentic RAG y justificación de decisiones técnicas.....3
Ejemplos de prompts y estrategias de rompt engineering.....4
Desafíos encontrados y soluciones implementadas.....5
Pruebas y resultados obtenidos.....6
Reflexion y posibles mejoras.....7

Arquitectura del agentic RAG y justificación de decisiones técnicas

El sistema implementado es un Agentic RAG diseñado para asistir en consultas sobre contenido de Netflix y papaers academicos de inteligencia artificial. A diferencia de un RAG tradicional lineal, este sistema utiliza un agente que decide dinamivamente cuando y como utilizar herramientas de busqueda.

Componentes principales:

- **LLM:**
    - Utilizamos el modelo *'Qwen/Qwen3-4B-Instruct'* a través de un endpoint HuggingFace. Lo elegimos por su capacidad de razonamiento eficiente en un tamaño relativamente contenido (4B parametros), permitiendo respuestas rapidas.
- **Embeddings:**
    - Utilizamos el modelo de embeddings *'sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2'*. Lo elegimos por ser también un modelo ligero y eficiente que no carga mucho la CPU. Ademas, soporta multiples lenguajes, lo cual es crucial para procesar consultas y documentos que, en nuestro caso, están en espanol.
- **Base de datos vectorial:**
    - Utilizamos la tecnologia de 'ChromaDB'. La elegimos ya que fue una de las recomendadas a principio del curso. Cuenta con persistencia local (sin necesidad de consultar servidores externos) y se integra nativamente con LangChain.
- **Pipeline RAG:**
    - **Input:**
        - **Netflix:** Carga desde un CSV y se convierte a documentos con metadatos.
        - **Papers:** Carga desde PDFs, utilizando chunking con 'RecursiveCharacterTextSplitter' con un tamaño de 800 y un solapamiento de 100.
    - **Recuperacion:**
        - Busqueda por similitud semantica (k=3 o k=5 segun la herramienta)
- **Agente:**
    - **Capacidades:**
        - **Toma de decisiones:** Evalúa el contexto y la consulta para decidir si usar una herramienta o responder directamente.
        - **Memoria:** Mantiene una ventana de los ultimos 5 intercambios para permitir seguimiento del contexto.
        - **Control de alucinaciones**: Verifica que las respuestas generadas estén respaldadas por la informacion recuperada.

Ejemplos de prompts y estrategias de rompt engineering

Utilizamos varias estrategias clave de prompt engineering:

- **Prompt de desicion (Routing):**
    - El agente utiliza un prompt estructurado para decidir que accion tomar, forzando una salida en formato JSON para facilitar el parsing pragmatico.

      "Eres un asistente que decide qué herramientas usar basándote en la conversación. [...] Responde SOLO con JSON válido: …"

    - **Estrategia:** *Few-shot prompting* implícito (instrucciones claras de formato) y *Task decomposition* (separar la decisión de la generación de respuesta).

- **Prompt de generacion con contexto:**
    - Para generar la respuesta final, se inyecta el historial y el resultado de la herramienta.

      "Eres un asistente experto en Netflix. Responde de manera natural y conversacional. [...] Información obtenida: {tool_result} [...] Si el usuario preguntó algo relacionado con conversaciones anteriores, refiérelo apropiadamente."

    - **Estrategia:** *Persona adoption* (“asistente experto”) y *Context injection*
- **Prompt de validacion:**
    - Se utiliza un prompt adicional para asegurar la veracidad de la respuesta.

      "Eres un validador de respuestas. Tu trabajo es verificar si la respuesta está basada ÚNICAMENTE en la información proporcionada. [...] Responde SOLO con JSON:..."

    - **Estrategia:** Self-Correction/Reflection (el modelo se evalúa a sí mismo para detectar inconsistencias)
- **System prompr restrictivo**
    - Se utiliza un prompt adicional para evitar respuestas fuera del dominio en el modo de chat directo.

      "Eres un asistente especializado ÚNICAMENTE en: 1. Contenido de Netflix... 2. Papers académicos... REGLA CRÍTICA: Si te preguntan sobre algo que NO esté en estos dos temas, di honestamente: ‘Lo siento, no tengo información…’"

    - **Estrategia:** Constraint enforcement

Desafíos encontrados y soluciones implementadas

Algunos de los desafíos con los que nos topamos a lo largo del proyecto fueron:

- **Datos duplicados en la base de datos:**
    - **Problema:** Al almacenar los datos del CSV en la base de datos vectorial, si este proceso se repetia (cosa que sucede al estar desarrollando el agente) y no se le asignaba ids manualmente a los documentos, se almacenaban nuevamente en la base de datos, resultando en documentos duplicados, lo que se traducia en respuestas incoherentes al querer listar por ejemplo peliculas de ciencia ficcion. Esto resultaba en una pelicula apareciendo más de una vez en el listado.
    - **Solucion:** Eliminamos la base de datos antes de crearla (si esta ya existe) y a su vez obtenemos los ids del DataFrame para luego asignarlos al momento de almacenar los datos, evitando así que Chroma le asigne ids aleatorios.
- **Alucinaciones en respuestas:**
    - **Problema:** El modelo tendia a inventar informacion cuando no encontraba datos en el contexto o al recibir preguntas fuera del domino.
    - **Solucion:** Implementamos ‘AgenticRAGWhithHallucinationControl’. Esto añade una capa de validacion donde el LLM revisa si su propia respuesta está relacionada y justificada por los documentos recuperados. Si no es el caso, se fuerza una respuesta informacion insuficiente.
- **Pérdida de contexto:**
    - **Problema:** El agente no mantenia el contexto de la conversacion, por lo que le era imposible referenciar a mensajes o respuestas anteriores.
    - **Solucion:** Integramos una memoria a corto plazo (‘conversation_history’) que se le inyecta al prompt de decision y generacion, permitiendo al modelo entender referencias sobre el contexto de la conversacion.

Pruebas y resultados obtenidos

Realizamos varias pruebas a medida que fuimos construyendo la solucion para ir verificando que cada paso de la construccion del agente fuera lo más correcto posible. Algunas de estas pruebas quedaron en el código final a modo ilustrativo de lo que fue el proceso de construccion del agente.
Además, para minimizar errores al momento de entender los prompts o cualquier otro problema que pueda surgir por errores en la escritura de estos (por mas minimos que sean), tratamos de que todos los prompts internos estuvieran escritos sin ningun error, ya sea de ortografia como de escritura.

- **Chatbot**
    - **Prueba de Búsqueda:** Consultas como "Busca películas de terror" devolvieron resultados relevantes del dataset.
    - **Prueba de Memoria:** A la pregunta de seguimiento "¿Cuál de esas es la más reciente?", el agente identificó correctamente las películas mencionadas anteriormente y seleccionó la correcta.
    - **Prueba de rechazo:** Preguntas sobre "Cien años de soledad" o "Lasaña vegetariana" fueron correctamente rechazadas con el mensaje estándar de limitación de dominio.

- **Resumen Automático (Papers)**
    - **Herramienta:** ‘summarize_paper_topic’.
    - **Prueba:** Se solicitó "Resumir el artículo 'Attention Is All You Need' en tres párrafos".
    - **Resultado:** El sistema recuperó chunks relevantes del paper, los agrupó y generó un resumen coherente donde explica el mecanismo de atención, las matrices Q, K, V y la diferencia con la atención aditiva.
    - **Observación:** El uso de chunks de 800 caracteres con solapamiento permitió capturar explicaciones técnicas completas sin cortar frases clave.

Reflexión y posibles mejoras

Al finalizar el proyecto y luego de realizar las pruebas correspondientes, podemos concluir que el sistema demuestra que la “inteligencia” de un sistema no depende únicamente del modelo y/o de su tamaño, sino de cómo este se integra en una arquitectura bien diseñada.

Al rodear al LLM con una infraestructura con memoria conversacional, herramientas especializadas y mecanismos de auto validacion, logramos potenciar las capacidades nativas del modelo para que se pueda asemejar (a una escala pequeña y con las limitacionoes que esto conyeva) a lo que sería un agente de chat convencional.

Posibles mejoras:

- **Manejo de errores de JSON:** Actualmente, si el LLM falla en generar un JSON válido, el agente falla o no usa la herramienta. Se podría implementar un parser más robusto o un mecanismo para reintentar el paso si este falla.
- **Búsqueda Híbrida:** Combinar búsqueda vectorial (semántica) con búsqueda por palabras clave mejoraría la precisión en búsquedas de nombres propios o términos técnicos exactos.
- **Reranking:** Añadir un paso de reranking después de la recuperación inicial para ordenar mejor los documentos antes de pasarlos al LLM.